객체지향 프로그래밍 고급

9강 : 참조관계와 가상함수

[복습]

다음은 나이와 몸무게를 멤버변수로 갖는 동물 클래스 Animal이다. Animal 클래스로부터 상속받으며 COLOR를 갖고 있는 Dog 클래스를 정의하고 출력하시오.

COLOR는 enum으로 정의(BLACK, BLUE, GREEN, YELLOW)

```
class Animal
{
protected:
    int age;
    int weight;
....
};
```

```
int main()
{
        Dog dog(3, 4, YELLOW);
        dog.ShowData();
        return 0;
}
```

나이:3, 몸무게:4, 색깔:노랑

[복습] 사원클래스 ver1.0

사원 클래스 Employee는 사번, 이름, 주소를 멤버변수로 갖는다.
Employee의 파생클래스인 Manager 클래스를 생성하고 부서(depart)와 월급(salary) 정보를 추가하라.
아래와 같이 사원들의 모든 정보를 출력하라.

사번 이름 주소 봉급
1. 강한자, 경기도 부천시, 250만원
2. 홍길동, 서울시 종로구, 280만원
2. 홍길동, 서울시 종로구, 200만원
3. 강호동, 서울시 서초구, 220만원

```
int main()
{
	Manager m[]= {Manager("1", "강한자", "경기도 부천시", 250, "인사과"),
	Manager("2", "홍길동", "서울시 종로구", 280, "연구실")
	};
	cout << "사번 이름   주소      봉급 n";
	for (int i=0; i<2; i++)
		m[i].ShowData();
	return 0;
}
```

[심화예제] 급여관리 시스템 1.0

다음은 OrageMedia라는 회사가 운영하는 급여관리 시스템이다.

- 직원의 고용형태는 정규직(Permanet)이며 급여의 계산 방식은 다음과 같다.
  - 정규직 : 연봉제, 매달의 급여가 정해져 있다.

급여관리 시스템의 기능은 아래와 같다.

- 직원을 등록한다.
- 이번 달에 지불해야 할 직원의 이름과 급여의 정보를 출력한다.
- 이번 달에 지불해야 할 급여의 총합을 출력한다.

직원의 정보는 아래와 같다.

- 이름
- 급여정보

데이터는 아래와 같다.
“Kim”, 100만원
“Hong”, 200만원
“Lee”, 150만원

클래스설계-컨트롤 클래스 구현

❖ 정규직 인사정보 클래스

PermanentWorker
char name[100]; int salary; 생성자(char* name, int money); GetPay(); ShowSalaryInfo();

-데이터 관리

- 직원의 인사정보(이름, 월급)

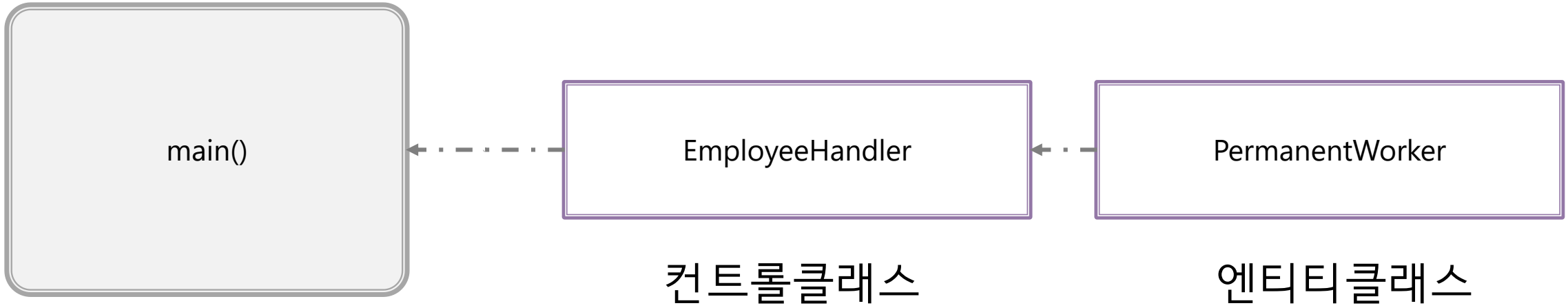
❖ 컨트롤 클래스

EmployeeHandler
PermanentWorker* empList[50]; int empNum; 생성자();; AddEmployee(PermanentWorker* emp); ShowAllSalaryInfo(); ShowTotalSalary(); 소멸자();

-기능 처리

- 새로운 직원 정보의 등록
- 모든 직원의 이번달 급여정보 출력
- 이번달 급여의 총액 출력

클래스설계-컨트롤 클래스 구현



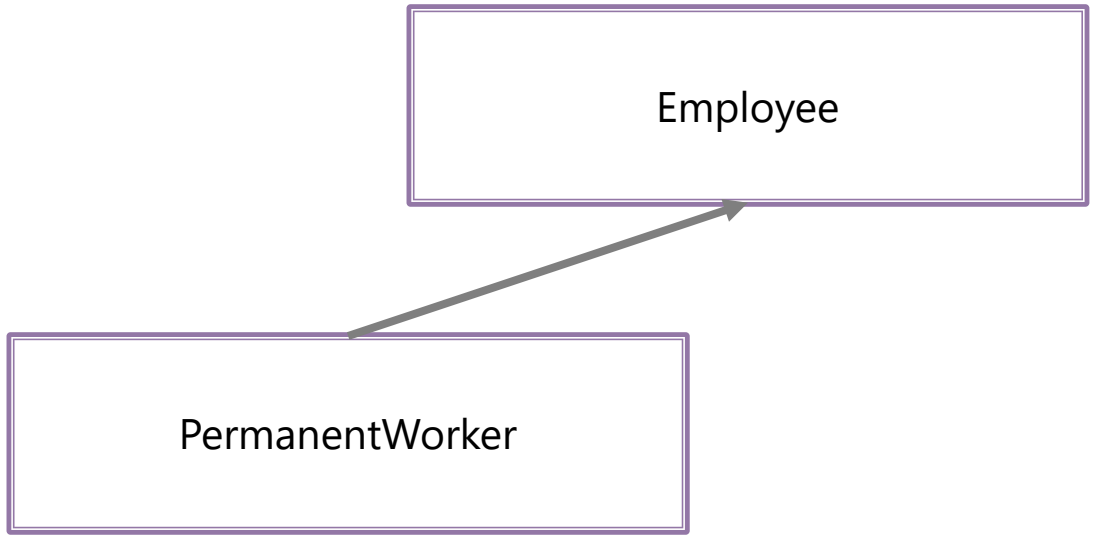
```
int main(void)
{
	// 직원관리를 목적으로 설계된 컨트롤 클래스의 객체생성
	EmployeeHandler handler;

	// 정규직 등록
	handler.AddEmployee(new PermanentWorker("JUN", 2000));
	handler.AddEmployee(new PermanentWorker("LEE", 3500));

	// 이번 달에 지불해야 할 급여의 정보
	handler.ShowAllSalaryInfo();

	// 이번 달에 지불해야 할 급여의 총합
	handler.ShowTotalSalary();
	return 0;
}
```

급여관리 시스템 ver1.1



다음은 OrageMedia라는 회사가 운영하는 급여관리 시스템이다.

- 직원의 고용형태는 현재는 정규직(Permanet)만 있으나 앞으로 영업직(Sales), 임시직(Temporary)으로 늘어날 계획이다.
- Employee 클래스를 설계하고 PermanentWorker클래스를 상속하라.
- Employee 클래스는 직원의 이름을 속성으로 갖는다.
- Employee 클래스의 GetPay()함수는 virtual로 선언하라

```
class Employee
{
private:
	char name[100];
public:
	Employee(char * name)
	{
		strcpy_s(this->name, strlen(name)+1, name);
	}
	void ShowYourName() const
	{
		cout << "name: " << name << endl;
	}
	virtual int GetPay() const
	{
		return 0;
	}
	virtual void ShowSalaryInfo() const
	{}
};
```

```
class PermanentWorker : public Employee
{
private:
	int salary;
public:
	PermanentWorker(char * name, int money)
	: Employee(name), salary(money)
	{}
	int GetPay() const
	{
		return salary;
	}
	void ShowSalaryInfo() const
	{
		ShowYourName();
		cout << "salary: " << GetPay() << endl;
	}
};
```

[과제]bank 3.0

뱅킹시스템에 컨트롤클래스(control class)를 적용해 본다.

✓ **컨트롤클래스**
- 프로그램의 전체 기능을 담당한다.(기능적 성격이 강한 클래스)
- 컨트롤 클래스만 봐도 프로그램의 전체 기능과 흐름을 파악할 수 있다.

✓ **엔티티클래스(Entity class)**
- 데이터적 성격이 강하다. 따라서 파일 및 데이터베이스에 저장되는 데이터를 소유하고 있다.
- 프로그램상에서 관리되는 데이터의 종류를 파악하는 데에 도움이 된다.

▪ **구현방법**
- 전역변수를 사용하지 않고 컨트롤 클래스를 이용한다.
- 컨트롤 클래스
  - AccountHandler라는 컨트롤 클래스를 정의하고 전역변수, 전역함수들을 멤버함수에 포함시킨다.
  - Account 객체의 저장을 위해 선언한 배열과 변수도 멤버변수에 포함시킨다.
  - AccountHandler 클래스 기반으로 프로그램이 실행되도록 main함수를 변경한다.

```
int main()
{
        AccoutHandler manager;
        int choice;

        while (1)
        {
                manager.ShowMenu();
                . . .
                switch(choice)
                {
                        case MAKE:
                                manager.MakeAccount();
                                break;
                        case DEPOSIT:
                                manager.DepositMoney();
                                break;
                        . . .
```

```
class AccountHandler
{
        Account *accArr[100];
        int accNum;
public:
        void MakeAccount();
        . . .
};
void AccountHandler::MakeAccount()
{
        . .
        cout << [계좌개설]" << endl;
        cout << "계좌ID";cin>>id;
        . . .
        accArr[accNum++] = new Account(id, balance, name);
}
```

Chapter 9

객체 포인터의 참조관계

가상함수

가상소멸자와 참조자의 참조 가능성

[예제1]

Car 클래스와 파생클래스 Automobile 클래스가 있다. (필요한 멤버함수는 추가로 정의) 아래와 같은 결과가 출력되도록 프로그램을 구현하라.(메인함수는 그대로 이용)

클래스	관계	멤버변수	멤버함수
Car	부모	string carNo, char * factory	Show()
Automobile	자식	string model	Show()

```
차량번호: 14다 4399
제조사: 기아
모델명: K5
----------------------
차량번호: 10가 1234
제조사: 기아
모델명: K7
----------------------
차량번호: 13가 5555
제조사: 기아
모델명: K7
----------------------
계속하려면 아무 키나 누르십시오 . . .
```

```
int main()
{
	Automobile *pt[3];
	pt[0] = new Automobile("14다 4399", "기아", "K5");
	pt[1] = new Automobile("10가 1234", "기아", "K7");
	pt[2] = new Automobile("13가 5555", "기아", "K9");
	for (int i = 0; i < 3; i++)
		pt[i]->Show();
	for (int i = 0; i < 3; i++)
		delete pt[i];
	return 0;
}
```

객체 포인터 변수

```
Person *ptr;                    // 포인터 변수 선언
ptr = new Person();             // 포인터 변수의 객체 참조
```

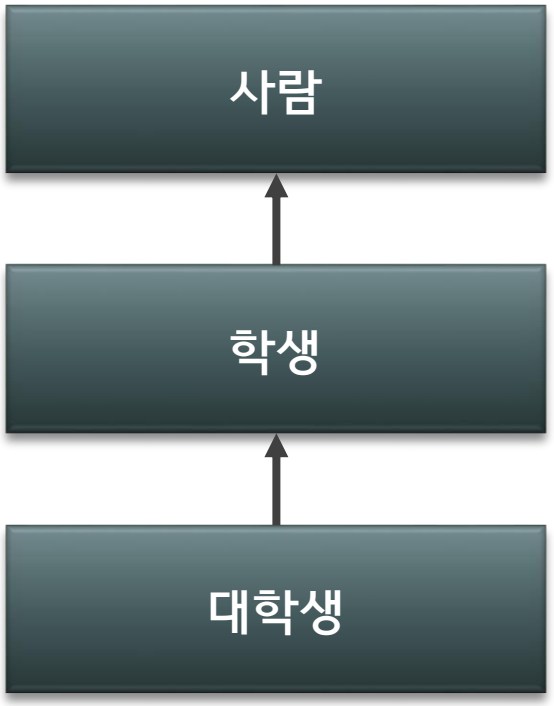
객체 포인터의 참조관계

→ 객체포인터 변수는 해당 객체 또는 객체를 직접 혹은 간접적으로 상속하는 모든 객체를 가리킬 수 있다.

```
class Person
{ …   };
class Student : public Person
{   . . . };
class UnivStudent : public Student
{   . . . };

Person *ptr = new Student();
Person *ptr = new UnivStudent();
Student *ptr = new UnivStudent();
```

객체 포인터의 참조관계



"학생은 사람이다."

"대학생은 학생이다."

"대학생은 사람이다."

"사람은 학생이다."

"학생은 대학생이다." X

"사람은 대학생이다."

객체 포인터의 참조관계

```
class Person
{
public:
    void Sleep() { cout << “Sleep” << endl; }
};

class Student : public Person
{
public:
    void Study() { cout << “Study” << endl; }
};

class UnivStudent : public Student
{
public:
    void Work() { cout << “Work” << endl; }
};
```

```
int main()
{
    Person * ptr1 = new Student();
    Person * ptr2 = new UnivStudent();
    Student * ptr3 = new UnivStudent();

    ptr1->Sleep();
    ptr2->Sleep();
    ptr3->Study();

    delete ptr1;
    delete ptr2;
    delete ptr3;

    return 0;
}
```

객체 포인터의 참조관계

→ 앞의 예제에서 Person 클래스의 객체 포인터인 ptr2가 UnivStudent 클래스의 함수를 가리킬 수 있는가?

```
int main()
{
    Person * ptr2 = new UnivStudent();

    ptr2->Study();

    delete ptr2;
    return 0;
}
```

→ Person 클래스에 Study()함수가 없으므로 컴파일 오류 발생

객체 포인터의 참조관계

```
int main()
{
    Student * sptr = new Student();
    Person *ptr = sptr;
}
```

파생클래스 파생클래스의 객체명;
기본클래스명 *객체명 = &파생클래스의 객체명

컴파일 결과는? **O**

객체 포인터의 참조관계

```
class First
{
public:
	void FirstFunc() { cout << "First Function" << endl; }
};

class Second : public First
{
public:
	void SecondFunc() { cout << "Second Func" << endl; }
};

class Third : public Second
{
public:
	void ThirdFunc() { cout << "Third Func" << endl; }
};
```

```
int main()
{
	Third *tptr = new Third();
	Second *sptr = tptr;
	First *fptr = sptr;

	tptr->FirstFunc();
	tptr->SecondFunc();
	tptr->ThirdFunc();

	sptr->FirstFunc();
	sptr->SecondFunc();
	sptr->ThirdFunc();

	fptr->FirstFunc();
	fptr->SecondFunc();
	fptr->ThirdFunc();

	return 0;
}
```

객체 포인터의 참조

- C++에서 AAA형 포인터 변수는 AAA 객체 또는 AAA를 직접/간접적으로 상속하는 모든 객체를 가리킬 수 있다.
- "포인터를 이용한 연산의 가능성 여부를 판단할 때, 포인터의 자료형을 기준으로 판단한다."

오버라이드(override) : 재정의

- **부모클래스에서 정의한 멤버함수를 자식클래스에서 변경하여 구현하는 것**
- **부모클래스에 있는 멤버함수를 자식클래스에서 재정의하면 자식클래스에서는 부모클래스에서 정의한 멤버함수가 무시되고 새로 정의한 함수가 동작한다.**
- **재정의는 다형성을 구현한다.**
- **이런 다형성을 구현하려면 부모클래스에 virtual 예약어를 붙여 가상함수로 정의한 후 자식클래스에서 재정의한다.**

cf. 오버로딩

같은 범위(함수, 클래스)내에 같은 이름과 다른 서명(함수명과 매개변수)을 가진 함수가 두 개 이상인 경우

함수의 오버라이딩

```
class First
{
public:
	void MyFunc()
		{ cout << "First Function" << endl; }
};
class Second : public First
{
public:
	void MyFunc()
		{ cout << "Second Func" << endl; }
};
class Third : public Second
{
public:
	void MyFunc()
		{ cout << "Third Func" << endl; }
};
```

```
int main()
{
	Third *tptr = new Third();
	Second *sptr = tptr;
	First *fptr = sptr;

	tptr->MyFunc();
	sptr->MyFunc();
	fptr->MyFunc();
	delete tptr;
	return 0;
}
```

함수의 오버라이딩

- 앞의 예제에서 객체 포인터 tptr, sptr, fptr이 호출하는 함수 MyFunc()은 누구의 것인가?

→ **포인터형 변수는 포인터의 자료형을 기준으로 판단하므로** 각 객체 포인터가 선언된 클래스의 함수를 가리킨다.

```
tptr->MyFunc(); → Third 클래스
sptr->MyFunc(); → Second 클래스
fptr->MyFunc(); → First 클래스
```

- **이것의 문제점은?**

함수의 오버라이딩

- 오버라이딩한 것은 해당 객체에서 호출되어야 하는 함수를 바꾼다는 의미임
- 따라서 포인터 변수의 자료형에 따라서 호출되는 함수의 종류가 달라지면 안됨
- 이를 해결하기 위한 방법은?

가상함수

- 부모클래스에 virtual 예약어를 붙여 가상함수로 정의한 후 자식클래스에서 재정의한다.
- 가상함수가 선언되면, 이 함수를 오버라이딩하는 함수도 가상함수가 된다.

```
class First
{
public:
	virtual void MyFunc() { cout << "First Function" << endl; }
};
```

가상함수

```
class First
{
public:
	virtual void MyFunc()
	{ cout << "First Function" << endl; }
};
class Second : public First
{
public:
	virtual void MyFunc()
	{ cout << "Second Func" << endl; }
};
class Third : public Second
{
public:
	virtual void MyFunc()
	{ cout << "Third Func" << endl; }
};
```

```
int main()
{
	Third *tptr = new Third();
	Second *sptr = tptr;
	First *fptr = sptr;

	tptr->MyFunc();
	sptr->MyFunc();
	fptr->MyFunc();
	delete tptr;
	return 0;
}
```

가상함수

"함수가 가상함수로 선언되면, 해당 함수 호출 시, 포인터의 자료형을 기반으로 호출대상을 결정하지 않고, 포인터 변수가 가리키는 객체를 참조하여 호출의 대상을 결정한다."

가상함수의 사용

```
class Animal
{
protected:
	int Age;
	int Weight;
public:
	void Speak(int count) {
		cout << count << "번 울부짖다.\n"; }
	virtual void Speak() { cout << "동물이 운다\n"; }
};

class Dog : public Animal
{
public:
	void wagTail() { cout << "꼬리를 흔들다\n"; }
	void Speak() {cout << "멍멍 짖다\n"; }
};
```

```
int main()
{
	Animal obj;
	Dog happy;
	obj.Speak();
	obj.Speak(3);

	happy.Speak();
	happy.Animal::Speak(5);

	return 0;
}
```

가상함수의 사용

- **가상함수는 부모클래스를 상속하는 자식클래스를 갖는 경우에서만 유효하다.**
- **참조나 포인터를 사용하여 호출한다.**

```
int main()
{
	Animal *pt;
	Dog happy;
	pt = &happy;
	pt->Speak();

	return 0;
}
```

- virtual함수이면 자식클래스의 함수가 호출
- virtual 함수가 아니면 부모클래스의 함수가 호출됨

가상함수의 사용

```
#include <iostream>
using namespace std;

class First
{
public:
    virtual void SimpleFunc() {
        cout << "First" << endl; }
};
class Second : public First
{
public:
    virtual void SimpleFunc() {
        cout << "Second" << endl;}
};
```

```
int main()
{
    First * ptr = new First();
    ptr->SimpleFunc();
    delete ptr;

    ptr = new Second();
    ptr->SimpleFunc();
    delete ptr;

    return 0;
}
```

→ **ptr은 동일한 포인터 변수이나, 실행 결과는 달라짐**

```
class First
{
public:
	void FirstFunc() { cout << "FirstFunc()" << endl;; }
	virtual void SimpleFunc()
	{ cout << "First SimpleFunc" << endl; }
};

class Second : public First
{
public:
	void SecondFunc() { cout << "SecondFunc()" << endl; }
	virtual void SimpleFunc()
	{ cout << "Second SimpleFunc" << endl; }

};

class Third : public Second
{
public:
	void ThirdFunc() { cout << "ThirdFunc()" << endl; }
	virtual void SimpleFunc()
	{ cout << "Third SimpleFunc" << endl; }
};
```

```
int main()
{
	Third obj;
	obj.FirstFunc();
	obj.SecondFunc();
	obj.ThirdFunc();
	obj.SimpleFunc();

	Second & sref = obj;
	sref.FirstFunc();
	sref.SecondFunc();
	sref.SimpleFunc();

	First & fref = sref;
	fref.FirstFunc();
	fref.SimpleFunc();

	return 0;
}
```

```
FirstFunc()
SecondFunc()
ThirdFunc()
Third SimpleFunc
FirstFunc()
SecondFunc()
Third SimpleFunc
FirstFunc()
Third SimpleFunc
계속하려면 아무 키나 누르십시오 . . .
```

[예제1]가상 함수

자동차 클래스와 이로부터 파생된 레이싱카 클래스를 정의하고 main()함수에서 객체포인터 배열을 이용하여 아래 데이터를 출력하시오.

Car 클래스
- 차번호(int num)
- 가솔린양(double gas)
- 차정보 출력(virtual Show())

RacingCar 클래스
- 주행코스(int course)
- 레이싱카정보 출력(virtual Show())

```
자동차의 번호 : 8034
자동차의 가솔린양 : 25.5

레이싱카의 번호 : 3455
레이싱카의 가솔린양 : 30.4
코스번호 : 3
```

```
Car *pCar[2] = { new Car(4352, 35.5), new RacingCar(4344, 32.1, 3) };
```

[예제2]

평수와 주소를 갖는 Building 클래스와 이로부터 상속받는 House 클래스, Office 클래스가 있다. 아래를 참고하여 클래스를 정의하고 Building 클래스의 객체포인터배열을 이용하여 데이터를 생성하시오.

Building 클래스
- int pyeong // 평수
- char * address // 주소

House 클래스
- int numrooms // 방의 개수
- int cost // 가격

Office 클래스
- string zoneCode // 우편번호
- int rent // 전세금

데이터
- House(32, "경기도 부천시 원미구", 4, 350000000)
- Office(24, "서울 종로구", "110-400", 250000000)
- Office(24, "인천 서구", "405-222", 200000000)

```
#include <iostream>
#include <string>
using namespace std;

class Building
{
protected:
	int pyeong;
	char * address;
public:
	Building(int py, char *adr);
	virtual ~Building() { delete[] address; }
	virtual void ShowData();
};
Building::Building(int py, char * adr) : pyeong(py)
{
	address = new char[strlen(adr)];
	strcpy_s(address, strlen(adr) + 1, adr);
}
void Building::ShowData()
{
	cout << "평수: " << pyeong << endl;
	cout << "주소 : " << address << endl;
}
```

```
class House : public Building
{
private:
	int numRooms;
	int cost;
public:
	House(int py, char* adr, int n, int c);
	void ShowData();
	~Building(){};
};
House::House(int py, char* adr, int n, int c)
: Building(py, adr), numRooms(n), cost(c) {}
void House::ShowData()
{
	cout << "평수: " << pyeong << endl;
	cout << "주소 : " << address << endl;
	cout << "방의개수: " << numRooms << endl;
	cout << "가격 : " << cost << endl;
	cout << "-------------------------\n";
}
```

```
class Office : public Building
{
	string zoneCode;
	int rent;
public:
	Office(int py, char* adr, string z, int rt);
	void ShowData();
};
Office::Office(int py, char* adr, string z, int rt)
: Building(py, adr), zoneCode(z), rent(rt) {}
void Office::ShowData()
{
	cout << "평수: " << pyeong << endl;
	cout << "주소 : " << address << endl;
	cout << "우편번호: " << zoneCode << endl;
	cout << "전세금 : " << rent << endl;
	cout << "-------------------------\n";
}
~Office() {}
```

```
int main()
{
	Building *build[3] = { new House(32, "경기도 부천시", 4,
	350000000),
	  new Office(24, "서울시 종로구", "110-444", 250000000),
	  new Office(24, "인천 서구", "404-500", 200000000) };

	for (int i = 0; i < 3; i++)
		build[i]->ShowData();
	return 0;
}
```